validation attribute. In this case, you would be able to use the attribute only in the application that defines `CurrencyConverterModel`.

Within the `IsValid()` method, you can cast the `value` provided to the required data type (in this case, `string`) and check against the list of allowed codes. If the code isn't allowed, the attribute returns a `ValidationResult` with an error message indicating that there was a problem. If the code *is* allowed, `ValidationResult.Success` is returned, and the validation succeeds.

Putting your attribute to the test in figure 32.6 shows that when `CurrencyTo` is an invalid value (£$%), the validation for the property fails, and an error is added to the `ModelState`. You could do some tidying-up of this attribute to set a custom message, allow nulls, or display the name of the property that's invalid, but all the important features are there.

